



طراحی سیستم‌های دیجیتال

بهار ۱۴۰۵

استاد: دکتر اجلائی

محسن صلاح - ۴۰۳۱۰۶۲۳۸

محاسبه گر سریال CRC-8

تمرین ۵ - سوال ۴

۱ هدف تمرین

در این تمرین یک محاسبه گر CRC-8-ATM سریال با چند جمله‌ای $P(x) = x^8 + x^2 + x + 1$ طراحی شده است. داده‌ی ورودی بیت به بیت و MSB-first از طریق `in_data` وارد مدار می‌شود و پس از پردازش `length_msg` بیت، مقدار نهایی روی `out_crc` قرار می‌گیرد و `valid` دقیقاً یک سیکل فعال می‌شود.

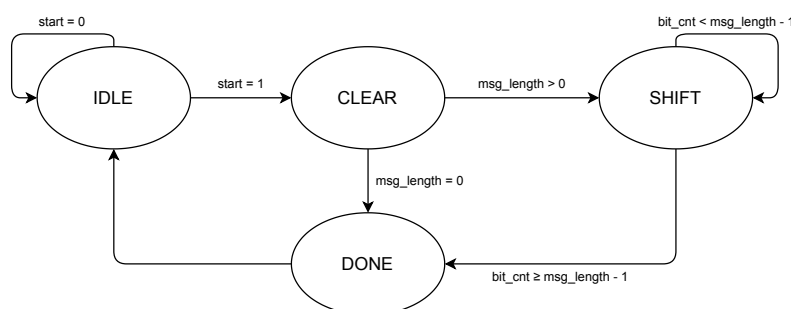
۲ توضیح معماری

- مدار از سه ماژول مجزا تشکیل شده که در یک ماژول سطح بالا به نام `crc8_top` با یکدیگر ترکیب می‌شوند:
- `d_ff`: فلیپ فلاپ D با ریست آسنکرون `active-low`، `synchronous clear`، و `enable`.
- `crc_datapath`: مسیر داده‌ی ساختاری شامل هشت نمونه از `d_ff` و گیت‌های XOR برای پیاده‌سازی چند جمله‌ای CRC-8-ATM.
- `crc_controller`: کنترل کننده‌ی رفتاری مبتنی بر FSM که با سیگنال‌های `en_shift` و `sync_clr`، مسیر داده را هدایت می‌کند.

وقتی `sync_clr = 1` است، تمام فلیپ فلاپ‌ها در لبه بالا رونده بعدی کلاک صفر می‌شوند. وقتی `en_shift = 1` است، ثابت CRC با محاسبه بیت جدید بروز رسانی می‌شود. خروجی `crc_out` مستقیماً سیم `crc_value` از مسیر داده است و هیچ ثابت میانی ندارد.

۱.۲ ماشین حالت کنترل کننده

کنترل کننده یک FSM با چهار حالت است:



شکل ۱: دیاگرام FSM کنترل کننده CRC-8

حالت فعلی	شرط انتقال	حالت بعدی	en_shift	sync_clr	valid
IDLE	start = 0	IDLE	۰	۰	۰
IDLE	start = 1	CLEAR	۰	۰	۰
CLEAR	length_msg = 0	DONE	۰	۱	۰
CLEAR	length_msg > 0	SHIFT	۰	۱	۰
SHIFT	bit_cnt < length_msg-1	SHIFT	۱	۰	۰
SHIFT	bit_cnt >= length_msg-1	DONE	۱	۰	۰
DONE	—	IDLE	۰	۰	۱

کنترل‌کننده FSM حالت جدول: Table ۱

ریست آسنکرون active-low (n_rst) نیز از هر حالتی کنترل‌کننده را بلافاصله به IDLE بازمی‌گرداند.

۳ توضیح کد

۱.۳ ماژول d_ff

هر بیت ثبات CRC توسط یک نمونه از این ماژول ذخیره می‌شود. اولویت‌بندی منطق به این صورت است: ریست آسنکرون بالاترین اولویت را دارد، سپس synchronous clear، و در نهایت enable:

```

1 always @(posedge clk or negedge rst_n) begin
2     if (!rst_n) q <= 1'b0;
3     else if (sync_clr) q <= 1'b0;
4     else if (en_shift) q <= d;
5 end

```

۲.۳ مسیر داده (crc_datapath)

بیت feedback و مقادیر بعدی ثبات به صورت کاملاً ترکیبی با assign تعریف شده‌اند. بیت‌های ۰، ۱ و ۲ به دلیل ضریب داشتن در چندجمله‌ای $P(x)$ از طریق XOR به feedback متصل می‌شوند:

```

1 wire feedback;
2 assign feedback = data_in ^ crc_value[7];
3
4 assign crc_next[7] = crc_value[6];
5 assign crc_next[6] = crc_value[5];
6 assign crc_next[5] = crc_value[4];
7 assign crc_next[4] = crc_value[3];
8 assign crc_next[3] = crc_value[2];
9 assign crc_next[2] = crc_value[1] ^ feedback;
10 assign crc_next[1] = crc_value[0] ^ feedback;
11 assign crc_next[0] = feedback;

```

سپس هشت نمونه از d_ff به صورت ساختاری نمونه‌سازی می‌شوند. دو نمونه اول به صورت زیر است و بقیه به همین شکل تا dff7 ادامه می‌یابند:

```

1 d_ff dff0 (.clk(clk), .rst_n(rst_n), .sync_clr(sync_clr),
2   .en_shift(en_shift), .d(crc_next[0]), .q(crc_value[0]));
3 d_ff dff1 (.clk(clk), .rst_n(rst_n), .sync_clr(sync_clr),
4   .en_shift(en_shift), .d(crc_next[1]), .q(crc_value[1]));
5 // dff2 dff7

```

۳.۳ کنترل‌کننده (crc_controller)

منطق next state و خروجی‌های کنترلی در دو بلاک @(*) always جداگانه نوشته شده‌اند. بلاک next state به صورت زیر است:

```

1 case (state)
2   IDLE: begin
3     if (start) next_state = CLEAR;
4     else next_state = IDLE;
5   end
6   CLEAR: begin
7     if (msg_length == 16'd0) next_state = DONE;
8     else next_state = SHIFT;
9   end
10  SHIFT: begin
11    if (bit_cnt >= msg_length - 1'b1) next_state = DONE;
12    else next_state = SHIFT;
13  end
14  DONE: next_state = IDLE;
15  default: next_state = IDLE;
16 endcase

```

وجود حالت CLEAR باعث می‌شود که ابهام زمانی بین دریافت start و صفر شدن ثبات CRC از بین برود. بلاک خروجی (Moore) به صورت زیر است:

```

1 case (state)
2   CLEAR: sync_clr = 1'b1;
3   SHIFT: en_shift = 1'b1;
4   DONE: valid = 1'b1;
5   default: ;
6 endcase

```

شمارنده bit_cnt در حالت CLEAR صفر شده و در هر سیکل SHIFT یک واحد افزایش می‌یابد. شرط خروج از SHIFT با عملگر >= نوشته شده تا در صورت هر گونه تخطی شمارنده، مدار گیر نکند.

task run_crc_test

تسک اصلی یک محاسبه کامل را اجرا و نتیجه را تأیید می‌کند. ساختار کلی آن به این صورت می‌باشد:

```

1 task run_crc_test;
2     input [127:0] msg_data;
3     input [15:0] len;
4     input trigger_busy_start;
5
6     reg [7:0] exp_crc;
7     reg [7:0] ref_crc; // running expected CRC for per-bit trace
8     integer i;
9     begin
10        exp_crc = calc_crc_stream(msg_data, len);
11
12        // assert start for 1 cycle, then wait for CLEAR to complete
13        start = 1'b1; msg_length = len;
14        @(posedge clk); start = 1'b0;
15        @(posedge clk);
16
17        if (len > 0) begin
18            ref_crc = 8'h00;
19            for (i = len - 1; i >= 0; i = i - 1) begin
20                data_in = msg_data[i];
21                @(posedge clk); #1;
22                ref_crc = calc_crc_bit(ref_crc, msg_data[i]);
23                $display(" [Bit %3d] data_in=%b | crc_reg=0x%02h (exp=0x%02h)",
24                    len-1-i, data_in, crc_out, ref_crc);
25            end
26        end
27
28        #1; // settle into DONE state
29        // check: valid === 1 && crc_out === exp_crc
30        // check: valid drops to 0 on next posedge (1-cycle pulse)
31    end
32 endtask

```

مدل مرجع نرم‌افزاری (calc_crc_bit و calc_crc_stream) همان منطق ثابت CRC را شبیه‌سازی می‌کند تا مقدار مورد انتظار به صورت خودکار محاسبه شود و نیازی به hardcode کردن مقادیر نباشد. در حین SHIFT، ref_crc بیت‌به‌بیت بروزرسانی شده و در کنار crc_out چاپ می‌شود. در نهایت بررسی می‌شود که valid دقیقاً یک سیکل فعال بماند.

تست کیس‌ها

قبل از شروع تست‌های اصلی، rst_n اعمال شده و سپس آزاد می‌شود تا مدار در وضعیت اولیه قرار گیرد:

```
1 // Test 1: Asynchronous Active-Low Reset
2 rst_n = 0; #2;
3 if (valid === 1'b0 && crc_out === 8'h00) ...
4 rst_n = 1;
```

rst_n در وسط سیکل ساعت (نه روی posedge) پایین می‌رود. بررسی بلافاصله و بدون انتظار برای لبه بعدی انجام می‌شود تا ناسنکرون بودن ریست تأیید شود. مقدار مورد انتظار 0x00 و crc_out = 0 و valid = 0 است.

```
1 // Test 2: Zero Message Length
2 run_crc_test(128'h0, 16'd0, 1'b0);
```

با length_msg = 0، FSM باید مستقیماً از CLEAR به DONE برود بدون اینکه هیچ سیکل SHIFT اجرا شود. مقدار مورد انتظار 0x00 است که همان مقدار اولیه ثبات CRC است.

```
1 // Test 3: Single-Bit Message (boundary: minimum non-zero length)
2 run_crc_test(128'h1, 16'd1, 1'b0);
```

کمترین طول غیر صفر ممکن. تنها یک سیکل SHIFT اجرا می‌شود و شرط خروج (bit_cnt >= length_msg) 1- بلافاصله برقرار است. بیت ارسال شده 1'b1 است و trace خروجی یک خط دارد.

```
1 // Test 4: Standard 8-bit message 0xA5
2 run_crc_test(128'hA5, 16'd8, 1'b0);
```

پیام استاندارد ۸ بیتی 0xA5 = 1010_0101 به صورت MSB-first ارسال می‌شود. trace کامل ۸ خطی چاپ می‌شود و مقدار نهایی crc_out با مدل مرجع مقایسه می‌شود.

```
1 // Test 5: 16-bit message 0x1234 (different length)
2 run_crc_test(128'h1234, 16'd16, 1'b0);
```

طول پیام دو برابر تست قبل است تا صحت شمارنده bit_cnt برای طول‌های بزرگ‌تر بررسی شود. مقدار 0x1234 شامل بیت‌های صفر پیشرو است که رفتار feedback را در حالت 0 = crc_reg[7] آزمایش می‌کند.

```
1 // Test 6: 24-bit message 0x8D2F31 (third distinct length)
2 run_crc_test(128'h8D2F31, 16'd24, 1'b0);
```

سومین طول متفاوت برای پوشش کامل‌تر بازه length_msg. پیام با بیت 1 شروع می‌شود (0x8D = 1000_1101) که از همان سیکل اول 1 = feedback می‌شود.

```
1 // Test 7: Busy Protection
2 run_crc_test(128'h3B1A, 16'd16, 1'b1);
```

آرگومان سوم 1'b1 است، یعنی تسک در وسط فاز SHIFT سیگنال start را مجدداً فعال می‌کند. چون FSM در حالت غیر IDLE است، این سیگنال نادیده گرفته می‌شود. درستی این رفتار با مقایسه crc_out نهایی با مدل مرجع — که بدون اطلاع از start محاسبه شده — تأیید می‌شود.

```
1 // Test 8: Consecutive Execution
2 run_crc_test(128'hFF00, 16'd16, 1'b0);
```

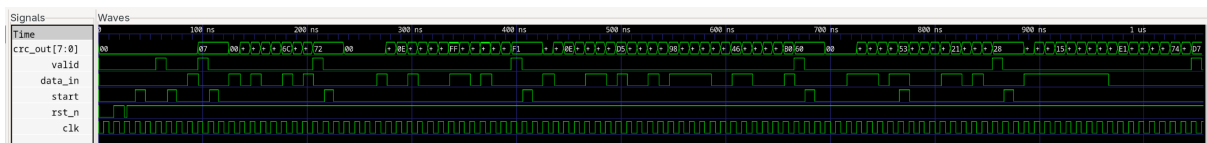
بلافاصله پس از تست ۷ و بدون ریست میانی اجرا می‌شود تا بررسی شود که مدار پس از DONE به IDLE بازگشته و آماده دریافت پیام بعدی است. پیام 0xFF00 شامل ۸ بیت یک و ۸ بیت صفر است و ثبات را از وضعیت اشباع کامل به حالت جدید می‌برد.

۴ اجرای کد

```
mohsen@Quanta:/mnt/D/SUT/Term_4/DSD/HW/HW-5/DSD_HWS_Mohsen_Salah_403106238/Q4$ iverilog DSD_HWS_Q4.v DSD_HWS_Q4_tb.v && vvp a.out
VCD info: dumpfile DSD_HWS_Q4.vcd opened for output.

=== Serial CRC-8 Calculator Testbench ===
[PASS] Test 1 (Async Reset): crc_out=0x00, valid=0
-----
Test 2: length_msg = 0 (boundary: CLEAR->DONE, CRC = 0x00)
[PASS] Test 2: Msg=0x00000000000000000000000000000000, Len=0 bits | CRC=0x00 (Exp=0x00), Valid=1-cycle
-----
Test 3: Single-bit message, bit=1 (boundary: length = 1)
--- SHIFT trace: 1 bit(s), MSB-first ---
Bit   data_in   crc_reg (DUT)   crc_reg (expected)
[Bit 0] 1       0x07       0x07
[PASS] Test 3: Msg=0x00000000000000000000000000000001, Len=1 bits | CRC=0x07 (Exp=0x07), Valid=1-cycle
-----
Test 4: Msg=0xAS, Len=8 bits
--- SHIFT trace: 8 bit(s), MSB-first ---
Bit   data_in   crc_reg (DUT)   crc_reg (expected)
[Bit 0] 1       0x07       0x07
[Bit 1] 0       0x0e       0x0e
[Bit 2] 1       0x1b       0x1b
[Bit 3] 0       0x36       0x36
[Bit 4] 0       0x6c       0x6c
[Bit 5] 1       0xdf       0xdf
[Bit 6] 0       0xb9       0xb9
[Bit 7] 1       0x72       0x72
[PASS] Test 4: Msg=0x00000000000000000000000000000a5, Len=8 bits | CRC=0x72 (Exp=0x72), Valid=1-cycle
-----
Test 5: Msg=0x1234, Len=16 bits
--- SHIFT trace: 16 bit(s), MSB-first ---
Bit   data_in   crc_reg (DUT)   crc_reg (expected)
[Bit 0] 0       0x00       0x00
[Bit 1] 0       0x00       0x00
[Bit 2] 0       0x00       0x00
[Bit 3] 1       0x07       0x07
[Bit 4] 0       0x0e       0x0e
[Bit 5] 0       0x1c       0x1c
[Bit 6] 1       0x3f       0x3f
[Bit 7] 0       0x7e       0x7e
[Bit 8] 0       0xfc       0xfc
[Bit 9] 0       0xff       0xff
[Bit 10] 1      0xfe       0xfe
[Bit 11] 1      0xfc       0xfc
[Bit 12] 0       0xff       0xff
[Bit 13] 1      0xfe       0xfe
[Bit 14] 0       0xfb       0xfb
[Bit 15] 0       0xf1       0xf1
[PASS] Test 5: Msg=0x0000000000000000000000000001234, Len=16 bits | CRC=0xf1 (Exp=0xf1), Valid=1-cycle
-----
```

شکل ۲: خروجی شبیه‌سازی در ترمینال



شکل ۵: شکل موج سیگنال‌ها در GTKWave

همان‌طور که مشاهده می‌شود، در هر اجرا `valid` دقیقاً یک سیکل فعال می‌شود، `out_crc` در همان سیکل مقدار صحیح را نشان می‌دهد، و سیستم در سیکل بعد به `IDLE` باز می‌گردد. همچنین در تست ریست، پایین رفتن `n_rst` بلافاصله و خارج از لبه کلاک، `out_crc` را صفر می‌کند.